HOMEWORK – 4

QUES 1: Implement a Stack using Queue data structure

CODE:

```
#include <stdio.h>

#define MAX 100

int q1[MAX], q2[MAX];
int f1 = 0, r1 = -1;
int f2 = 0, r2 = -1;

void push(int x)
{
   q2[++r2] = x;

   while (f1 <= r1)
      q2[++r2] = q1[f1++];

   f1 = 0;
   r1 = -1;

   while (f2 <= r2)
      q1[++r1] = q2[f2++];

   f2 = 0;
   r2 = -1;
}

void pop()
{
   if (f1 > r1)
```

```
        printf("Stack is empty\n");
    else
        printf("Popped: %d\n", q1[f1++]);
}

void display()
{
    int i;

    if (f1 > r1)
    {
        printf("Stack is empty\n");
        return;
    }

    for (i = f1; i <= r1; i++)
        printf("%d ", q1[i]);

    printf("\n");
}

int main()
{
    push(10);
    push(20);
    push(30);

    printf("Stack: ");
    display();

    pop();
```

```
    printf("Stack after pop: ");
    display();

    return 0;
}
```

QUES 2: Implement a Queue using Stack data structure

COED:

```
#include <stdio.h>

#define MAX 100

int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;

void enqueue(int x)
{
    s1[++top1] = x;
}

void dequeue()
{
    int x;

    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is empty\n");
        return;
    }
```

```
    if (top2 == -1)
    {
        while (top1 != -1)
            s2[++top2] = s1[top1--];
    }

    x = s2[top2--];

    printf("Deleted: %d\n", x);
}

void display()
{
    int i;

    printf("Queue: ");

    for (i = 0; i <= top2; i++)
        printf("%d ", s2[i]);

    for (i = 0; i <= top1; i++)
        printf("%d ", s1[i]);

    printf("\n");
}

int main()
{
    enqueue(10);
    enqueue(20);
```

```
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}
```

QUES 3: Write a C program using stack data structure to implement managing the history of web pages visited

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

char history[MAX][100];
int top = -1;

void visit(char page[])
{
    if (top == MAX - 1)
    {
        printf("History is full\n");
        return;
    }
```

```
    strcpy(history[++top], page);
}

void back()
{
    if (top == -1)
    {
        printf("No history available\n");
        return;
    }

    printf("Going back from: %s\n", history[top]);
    top--;

    if (top >= 0)
        printf("Current page: %s\n", history[top]);
    else
        printf("No page available\n");
}

void display()
{
    int i;

    printf("Browser History:\n");

    for (i = top; i >= 0; i--)
        printf("%s\n", history[i]);
}

int main()
```

```
{
   visit("google.com");
   visit("youtube.com");
   visit("github.com");

   display();

   back();
   back();

   return 0;
}
```

QUES4: Write a C program to sort a stack using temporary stack

CODE:

```
#include <stdio.h>

#define MAX 100

int stack[MAX], temp[MAX];
int top = -1, ttop = -1;

void push(int x)
{
   stack[++top] = x;
}

int pop()
{
   return stack[top--];
```

```
}

void sortStack()
{
   int x;

   while (top != -1)
   {
      x = pop();

      while (ttop != -1 && temp[ttop] > x)
      {
         stack[++top] = temp[ttop--];
      }

      temp[++ttop] = x;
   }

   while (ttop != -1)
      stack[++top] = temp[ttop--];
}

void display()
{
   int i;

   for (i = top; i >= 0; i--)
      printf("%d ", stack[i]);

   printf("\n");
}
```

```
int main()
{
   push(30);
   push(10);
   push(50);
   push(20);
   push(40);

   printf("Original stack: ");
   display();

   sortStack();

   printf("Sorted stack: ");
   display();

   return 0;
}
```

QUES 5: The stock span problem is a financial problem where we have a series of daily price quotes for a stock denoted by an array and the task is to calculate the span of the stocks price for all days. The span of a stocks price for ith day gives the maximum number of consecutive days leading upto ith days(including current day) where stocks price is less than equal to it's price on day i.

CODE:

```
#include <stdio.h>

#define MAX 100

int main()
{
```

```
int price[MAX], span[MAX], stack[MAX];
int n, top = -1;
int i;

printf("Enter number of days: ");
scanf("%d", &n);

printf("Enter stock prices:\n");

for (i = 0; i < n; i++)
   scanf("%d", &price[i]);

for (i = 0; i < n; i++)
{
   while (top != -1 && price[stack[top]] <= price[i])
      top--;

   if (top == -1)
      span[i] = i + 1;
   else
      span[i] = i - stack[top];

   stack[++top] = i;
}

printf("Stock Span:\n");

for (i = 0; i < n; i++)
   printf("%d ", span[i]);

return 0;
```

```
}
```

QUES 6: Given an array for each elements find the nearest element to it's right that has a higher frequency than the current element. If no such elements exists set value to -1. Implement using stack.

CODE:

```
#include <stdio.h>

#define MAX 100

int main()
{
   int arr[MAX], freq[MAX];
   int stack[MAX], result[MAX];
   int n, top = -1;
   int i, j;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("Enter array:\n");

   for (i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   for (i = 0; i < n; i++)
   {
      freq[i] = 0;

      for (j = 0; j < n; j++)
      {
         if (arr[i] == arr[j])
```

```
            freq[i]++;
        }
    }

    for (i = n - 1; i >= 0; i--)
    {
        while (top != -1 && freq[stack[top]] <= freq[i])
            top--;

        if (top == -1)
            result[i] = -1;
        else
            result[i] = arr[stack[top]];

        stack[++top] = i;
    }

    printf("Output:\n");

    for (i = 0; i < n; i++)
        printf("%d ", result[i]);

    return 0;
}
```

QUES 7: Given a pattern containing only I's and D's. I for increasing and D for decreasing. Device an algorithm using stack to print minimum numbers following that pattern. Digits from 1-9 and digits can't repeat

CODE:

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 100

int main()
{
   char pattern[MAX];
   int stack[MAX];
   int top = -1;
   int num = 1;
   int i;

   printf("Enter pattern containing I and D: ");
   scanf("%s", pattern);

   for (i = 0; i <= strlen(pattern); i++)
   {
      stack[++top] = num++;

      if (i == strlen(pattern) || pattern[i] == 'I')
      {
         while (top != -1)
            printf("%d", stack[top--]);
      }
   }

   return 0;
}
```

QUES 8: Given an array of digits (values are from 0 to 9) find the minimum possible sum of two numbers formed from digits of the array. All digits of given array must we used to form the two numbers use queue for implementation.

CODE:

```
#include <stdio.h>

int main()
{
   int arr[100], n;
   int i, j, temp;
   int num1 = 0, num2 = 0;

   printf("Enter number of digits: ");
   scanf("%d", &n);

   printf("Enter digits:\n");

   for (i = 0; i < n; i++)
      scanf("%d", &arr[i]);


   for (i = 0; i < n - 1; i++)
   {
      for (j = i + 1; j < n; j++)
      {
         if (arr[i] > arr[j])
         {
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
         }
      }
   }
```

```
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
            num1 = num1 * 10 + arr[i];
        else
            num2 = num2 * 10 + arr[i];
    }

    printf("Minimum sum = %d\n", num1 + num2);

    return 0;
}
```

QUES 9: Given an array and a positive integer k, find the first negative integer for each window (contiguous subarray) of size k. If a window does not contain a negative integer, then print 0 for that window. Implement using queue.

CODE:

```
#include <stdio.h>

#define MAX 100

int main()
{
    int arr[MAX], queue[MAX];
    int n, k;
    int front = 0, rear = -1;
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
    printf("Enter array:\n");

    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter window size: ");
    scanf("%d", &k);

    for (i = 0; i < n; i++)
    {
        if (arr[i] < 0)
            queue[++rear] = i;

        if (front <= rear && queue[front] <= i - k)
            front++;

        if (i >= k - 1)
        {
            if (front <= rear)
                printf("%d ", arr[queue[front]]);
            else
                printf("0 ");
        }
    }

    return 0;
}
```

QUES 10: Given a matrix of dimension m*n where each cell in the matrix can have values 0,1 and 2 which has the following meaning

0: empty cell

1: cell having fresh oranges

2: cells having rotten oranges

The task is to find the minimum to all required so that all oranges become rotten. A rotten oranges at index (I,j) can rot other fresh oranges which are it's neighbours (up, down, left, right). If impossible to rot every oranges then return -1 use queue for implementation.

CODE:

```
#include <stdio.h>

#define MAX 100

struct Node
{
   int row;
   int col;
};

int main()
{
   int arr[MAX][MAX];
   struct Node queue[MAX * MAX];

   int m, n;
   int front = 0, rear = -1;
   int i, j;
   int time = 0;
   int fresh = 0;

   int dr[] = {-1, 1, 0, 0};
   int dc[] = {0, 0, -1, 1};
```

```
printf("Enter rows and columns: ");
scanf("%d %d", &m, &n);

printf("Enter matrix:\n");

for (i = 0; i < m; i++)
{
   for (j = 0; j < n; j++)
   {
      scanf("%d", &arr[i][j]);

      if (arr[i][j] == 2)
      {
         queue[++rear].row = i;
         queue[rear].col = j;
      }

      if (arr[i][j] == 1)
         fresh++;
   }
}

while (front <= rear && fresh > 0)
{
   int size = rear - front + 1;

   for (i = 0; i < size; i++)
   {
      int r = queue[front].row;
      int c = queue[front].col;
      front++;
```

```
            for (j = 0; j < 4; j++)
            {
                int nr = r + dr[j];
                int nc = c + dc[j];

                if (nr >= 0 && nr < m &&
                    nc >= 0 && nc < n &&
                    arr[nr][nc] == 1)
                {
                    arr[nr][nc] = 2;
                    fresh--;

                    queue[++rear].row = nr;
                    queue[rear].col = nc;
                }
            }
        }

        time++;
    }

    if (fresh == 0)
        printf("Minimum time = %d\n", time);
    else
        printf("-1\n");

    return 0;
}
```